

Game Description (Rules)

1. The game board consists of **two rows of pits**:
 - Each row has **6 pits** for collecting stones.
 - Each player also has a **Store** (one for Player and one for Computer).
2. At the start, each pit contains **4 stones** (configurable).
3. Players take turns:
 - On a turn, the player selects one of their pits and distributes its stones **counterclockwise** into subsequent pits, including their **Store**, but **skipping the opponent's Store**.
 - If the last stone lands in the player's **Store**, the player gets another turn.
 - If the last stone lands in an **empty pit on their side**, the player captures that stone and all stones in the opposite pit.
4. The game ends when one side of the board is empty. The opponent collects remaining stones.
5. Winner: The player with the most stones in their **Store** wins.

Assignment Tasks

Students must implement the following in **C**:

1. Board Representation

- Represent the Game board using arrays.
- Clearly differentiate between **user pits, computer pits, and stores**.

2. Display Function

- Write a function to **print the board state** after each move in a user-friendly format.
- Example layout:

	[C6]	[C5]	[C4]	[C3]	[C2]	[C1]	
[Comp Store]							[User Store]
	[U1]	[U2]	[U3]	[U4]	[U5]	[U6]	

3. User Move

- Prompt the user to select a pit.
- Validate input (must be a non-empty pit on their side).
- Distribute stones according to game rules.

4. Computer Move (Basic AI)

- The computer should select its move automatically.
- Implement a **greedy strategy** (choose a pit that maximizes immediate stones captured or ensures an extra turn). Use Game Trees and Prunings.

5. Rule Handling

- Correctly handle:
 - Extra turns when last stone lands in own store.
 - Capture rule (empty pit + opposite pit).
 - End of game condition.

6. Winner Declaration

- When game ends, display **final scores** and declare the winner.

Possible Extensions

- Implement a **smarter AI** using look-ahead strategies.
- Allow **custom number of pits** or **stones per pit**.
- Provide a **menu system** with options like: New Game, Exit, Rules.
- Add **save and load** functionality using file handling in C.

Submission Guidelines

- Submit a single `.c` source file named `rollnumber_assignment1.c`.
- The program must compile with `gcc` without errors or warnings.
- Code should follow good programming practices:
 - Proper **indentation**
 - **Meaningful variable and function names**
 - **Comments** explaining logic